

Maze Solver Robot

Motor & Encoder Module — JGA25-370 DC12

Decisions, reasoning, sourced datasheet facts, edge cases, wiring, chassis placement and PCB guidance

Item	Detail
Course	EE4360 — Embedded Systems Design and Programming
Modules covered	2 × DC encoder motors (JGA25-370, DC12) and their H-bridge drivers
Document version	v1.0 — 26 September 2026
Primary sources	Official Maze Solver project specification, plus manufacturer/supplier datasheets for the JGA25-370 (cited in Section 3)
Companion documents	Power Supply System Design; Ultrasonic Module Design (this document corrects a pin conflict with that one — see the callout in Section 4)
Status	Design proposal — items in Section 4 (gear ratio, exact driver model) must be confirmed once the real hardware is in hand

How to read this document

Every important statement is tagged so you can tell where it comes from:

- **[SPEC]** — stated in the official project specification.
- **[DATASHEET]** — a figure from a manufacturer/supplier datasheet or product page for the actual part, not the university spec and not our guess — cited with its source.
- **[ASSUMPTION]** — something we assumed because neither the spec nor a datasheet tells us yet. Must be checked.
- **[RECOMMENDATION]** — our own engineering recommendation. Not a rule; it can be changed with good reason.

Motors and encoders are covered in **one** document because they are the same physical unit, wired to the same driver, and tuned together in a single control loop. The MPU-6050 is a separate I2C sensor with its own mounting and calibration concerns, so it has its own document.

1. Summary of the design

Each wheel is driven by a JGA25-370 DC12 gear motor with a built-in Hall-effect quadrature encoder, through one channel of the provided H-bridge driver. The driver's PWM (speed) and IN1/IN2 (direction) pins are on the Arduino Mega; encoder Channel A on each motor goes to a true interrupt pin for reliable pulse counting, with Channel B available on an ordinary pin so the option of full quadrature (direction) decoding is open without rewiring. Speed control is **closed-loop**: a PID controller reads live encoder ticks and adjusts PWM per wheel, rather than relying on a fixed PWM ratio between motors, so it self-corrects for motor mismatch, battery voltage sag, and load changes (including the bridge climb).

Decision	Choice	Main reason
Driver interface	PWM (ENA/ENB) + 2 direction pins per motor (IN1–IN4)	Matches the provided H-bridge driver style already assumed in the power system document
Encoder decoding	Channel A on a true interrupt pin now; Channel B wired but read only if/when quadrature is needed	Matches the team's existing working convention while keeping the upgrade path open at no extra wiring cost
Speed control	Closed-loop PID per wheel using encoder ticks/time, not a fixed PWM ratio	Self-corrects for motor mismatch, battery sag and load — a static ratio does not
Pin map	Motors + encoders on D2, D3, D5–D12 (Section 4.2)	Frees D18/D19 and moves ultrasonic ECHO off D2/D3/D18 — resolves a real conflict with the encoder pins
Direction-change handling	Bring PWM to 0 before flipping IN1/IN2, then ramp back up	Avoids H-bridge shoot-through and the current spike already flagged in the power document
Noise suppression	100 nF across each motor's terminals; encoder signal wires bundled apart from motor power wires	DC motor brush noise otherwise couples onto nearby encoder wiring

2. Requirements from the specification

These are the only hard rules that touch the motors and encoders. Everything else is derived. [\[SPEC\]](#)

Requirement	What the spec says	Effect on this design
Provided parts	2 DC encoder motors, 2 wheels, 2 metal wheel/motor brackets, 2 H-bridge motor drivers (p.6)	Fixes the count; no model number or driver IC is named
Mandatory parts	Must use the university's motors, wheels and driver, or be disqualified (p.2)	We integrate the provided hardware — no substitution
Extra equipment	Anything not on the provided list needs prior instructor approval (p.7)	Suppression capacitors, connectors, mounting hardware etc. should be cleared with the instructor
Size	Max 18.5 × 18.5 cm, 18 cm high (p.2)	Motor + bracket + wheel assembly and its wiring must fit inside this footprint
Bridge	Robot must climb an inclined bridge (< 35°) autonomously (p.2, p.7)	Sets the torque-margin check in Section 4.3
Autonomy	No human aid; no human touch once the activation button is pressed (p.2, p.6)	Motor control and stall recovery must be fully autonomous — no manual reset mid-run
Arena surface	Painted plywood; friction not guaranteed; floor seams may snag low parts (p.3)	Feeds the wheel-slip and traction discussion in Section 4.4

3. Confirmed hardware facts — JGA25-370 DC12

None of this is in the university spec, which only says “DC encoder motors.” These figures come from manufacturer/supplier listings for the JGA25-370 family and are the most reliable numbers available before the real units are measured on the bench (Section 9). [\[DATASHEET\]](#)

Property	Value	Source
Motor + gearbox	25 mm diameter “370” DC motor with metal spur-gear reduction	lk-tronics.com
Rated voltage	12 V DC (the “DC12” you confirmed)	lk-tronics.com
No-load current	≈ 0.1–0.3 A per motor, depending on gear ratio	lk-tronics.com; DigiKey/Seeed
Stall current	≈ 1.2–2.2 A per motor at 12 V	lk-tronics.com; DigiKey/Seeed
Stall torque (gearbox output)	≈ 5–20 kgf·cm, depending on gear ratio	lk-tronics.com; DigiKey/Seeed
Gear ratio	Commonly 1:10 to 1:150+ in this family — not stated for the university's units; read it off the metal gearbox	lk-tronics.com
Encoder type	Hall-effect magnetic, 2-channel quadrature (A/B)	abra-electronics.com
Encoder resolution	11 pulses/rev of the motor shaft , before the gearbox	abra-electronics.com
Encoder supply	3.3–5 V DC	abra-electronics.com
Motor cable	6-wire JST-PH: red/white = motor power, blue = encoder VCC, black = encoder GND, yellow = ch A, green = ch B	abra-electronics.com
Output shaft	4 mm, round or D-shaped	lk-tronics.com
Weight	≈ 150–200 g per motor (≈ 300–400 g for both)	lk-tronics.com

Cross-check with the power system document: that document assumed ≈ 0.25 A running and ≈ 2 A stall per motor. These sourced figures (0.1–0.3 A / 1.2–2.2 A) confirm that assumption was reasonable and tighten the range slightly — no change needed to the battery or fuse sizing there.

4. Assumptions, pin plan and a correction to the ultrasonic document

4.1 Open items to confirm

#	Assumption	If it is wrong...	How to check
A1	Driver is an L298N-style module: PWM speed pin + 2 direction pins per motor	A different driver (e.g. TB6612, BTS7960) may combine speed+direction differently	Read the driver IC printed on the module
A2	Gear ratio is unknown; worked examples use 1:50 as an illustrative mid-range value	The real ratio changes counts/revolution (Section 5.2) and available torque (Section 4.3)	Read the ratio printed on the metal gearbox, or count motor-shaft turns per one wheel turn by hand
A3	Wheel diameter $\approx$ 65 mm (matches the earlier power-document assumption)	A different wheel size changes mm/count and the torque-margin numbers	Measure the provided wheel directly
A4	Robot mass $\approx$ 1 kg (carried over from the power document)	A heavier finished robot needs more climbing torque than Section 4.3 shows	Weigh the finished chassis
A5	Starting decoding scheme is single-edge on Channel A only (matches the team's existing encoder code), not full quadrature	If backward wheel slip/collision detection becomes important, upgrade to quadrature (Section 5.2) — no rewiring needed, only firmware	Team decision, revisit if odometry drifts unexpectedly during testing

4.2 Master pin map, and a correction to the ultrasonic document

The team's own existing guides already fix **encoder Channel A on pins D2 and D3** (the Arduino Mega's two easiest interrupt pins), used consistently across several of your project docs. The earlier *Ultrasonic Module Design* document independently put ultrasonic ECHO on D2, D3 and D18 — that **conflicts** with the encoders. Encoders win that conflict: missing an encoder pulse corrupts odometry silently, while the ultrasonic Tier-1 design (`pulseIn()`) never needed a true interrupt pin in the first place. **[RECOMMENDATION]** Update the ultrasonic wiring to the corrected pins below.

Pin	Assigned to	Note
D2 (INT0)	Left encoder — Channel A	True interrupt; unchanged from the team's existing code
D3 (INT1)	Right encoder — Channel A	True interrupt; unchanged from the team's existing code
D5 (PWM)	Left motor speed (ENA)	New — was informally “pin 3” in an early teaching example; moved off D3 to avoid the encoder conflict
D6 (PWM)	Right motor speed (ENB)	New
D7 / D8	Left motor direction (IN1 / IN2)	New
D9 / D10	Right motor direction (IN3 / IN4)	New
D11	Left encoder — Channel B	New; ordinary pin, sampled inside the Channel-A ISR (Section 5.2) — no interrupt needed
D12	Right encoder — Channel B	New; same as above
D20 (INT3 / SDA)	MPU-6050 — I2C data	Fixed by the Mega's hardware I2C bus (IMU document)
D21 (INT2 / SCL)	MPU-6050 — I2C clock	Fixed by the Mega's hardware I2C bus (IMU document)
D18 (INT5), D19 (INT4)	Spare interrupt-capable pins	Freed up by this correction — available if ultrasonic Tier-2 (interrupt-based) timing is adopted later
D22-D24	Ultrasonic TRIG (Front/Left/Right)	Unchanged from the ultrasonic document
D25-D27	Ultrasonic ECHO (Front/Left/Right)	Corrected from D2, D3, D18 — ordinary pins are sufficient for the Tier-1 <code>pulseIn()</code> design
A0-A7	IR line array	Unchanged

Pin	Assigned to	Note
A15	Battery voltage divider	Unchanged, from the power document

4.3 Bridge-climb torque check

Force needed to climb (whole robot, ~1 kg assumed):

$$F = m \cdot g \cdot \sin(35^\circ) \approx 5.6 \text{ N}$$

Torque needed per driven wheel ($\varnothing$ 65 mm):

$\approx 0.9 \text{ kgf}\cdot\text{cm}$ — compare against the motor's

output-shaft torque at YOUR gear ratio

(Section 3) before assuming it will climb.

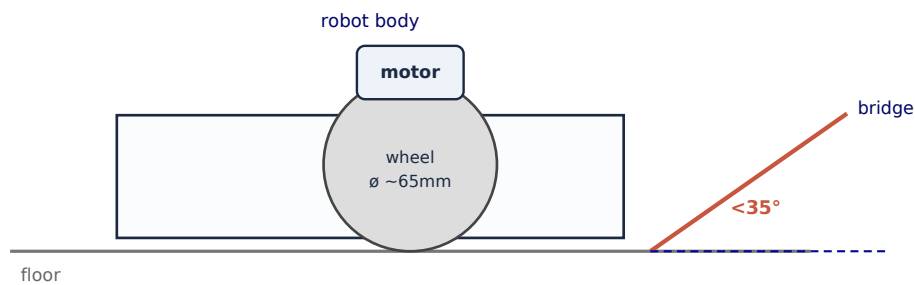


Figure 1 — Force and torque needed to climb the bridge, worked with the assumed mass and wheel size.

Using the assumed 1 kg robot mass (A4) and 65 mm wheels (A3), the force needed to climb a 35° incline is $F = m \cdot g \cdot \sin(35^\circ) \approx 1 \times 9.81 \times 0.574 \approx 5.6 \text{ N}$. Split evenly across the two driven wheels (ignoring the free-spinning casters and rolling resistance), that is about 2.8 N per wheel, or roughly **0.9 kgf·cm of torque at the wheel** (output shaft, after the gearbox).

Compared against the datasheet's 5–20 kgf·cm stall torque range (Section 3), there is large margin at most gear ratios — but that range spans a factor of 4, and a very low-reduction, high-speed variant of this motor family sits toward the low end. **This is a real risk, not a formality:** if the university's units turn out to be a high-speed/low-torque variant, the margin could be thin once friction, acceleration, and an underestimated robot mass are added. Test the actual bridge climb early, at realistic PWM, rather than assuming the datasheet range covers it.

4.4 Current, torque and edge-case analysis

Scenario	Cause	Effect if unhandled	Mitigation
Motor-to-motor mismatch	Manufacturing tolerance; even “identical” motors differ ~5–10%	Robot curves during “straight” commands	Closed-loop per-wheel PID (Section 5.3), not a fixed ratio
Dead zone	Static friction exceeds torque below $\approx$ PWM 90–120	Small PID corrections near zero speed do nothing	Measure and compensate per motor (existing team guide); PID output should skip straight through the dead zone, not linger in it
Stall (wall collision, wedge, jam)	Commanded PWM continues while wheel is physically blocked	Near-continuous stall current (Section 3) heats the motor/driver; encoder count stays flat while PWM stays high	PWM-high-but-ticks-flat for $>\sim 300$ ms $\rightarrow$ cut motors, enter a fault/backup state (already used for ultrasonic-driven stall logic; same signal here)
H-bridge shoot-through	Direction pins flipped while PWM is still active	Driver heating, possible long-term damage	PWM to 0 before changing direction, then ramp (Section 5.1)
Wheel slip (low traction, the bridge incline)	Torque exceeds available friction; wheel spins without the robot moving	Encoder over-reports true distance travelled — dead-reckoning error	Compare left/right encoder agreement during straight-line commands; a sudden divergence signals one side slipping (already used in the team's sensor-fusion notes)
Battery voltage sag	Pack voltage drops over the 30-minute access window (power document)	Same PWM gives less speed late in testing than early on	Closed-loop PID (Section 5.3) compensates automatically — an open-loop PWM table does not
Encoder noise / missed pulses	DC motor brush noise couples onto nearby signal wiring	Ticks undercount or jump, corrupting speed/odometry	100 nF motor-terminal capacitor + wiring separation (Section 5.5)
Loose motor connector	JST-PH connector shaken loose by vibration/bumps over a run	Motor or encoder cuts out mid-run — a real single-point failure	Secure with a zip-tie/strain relief and a small service loop (Section 7 placement rules)
Reversal transient	Fast direction change (line-following correction, bridge exit)	Brief high current (power document) and an ambiguous direction reading if using PWM-sign only	Same coast-to-zero-then-reverse rule fixes both problems at once

4.5 PWM response curve

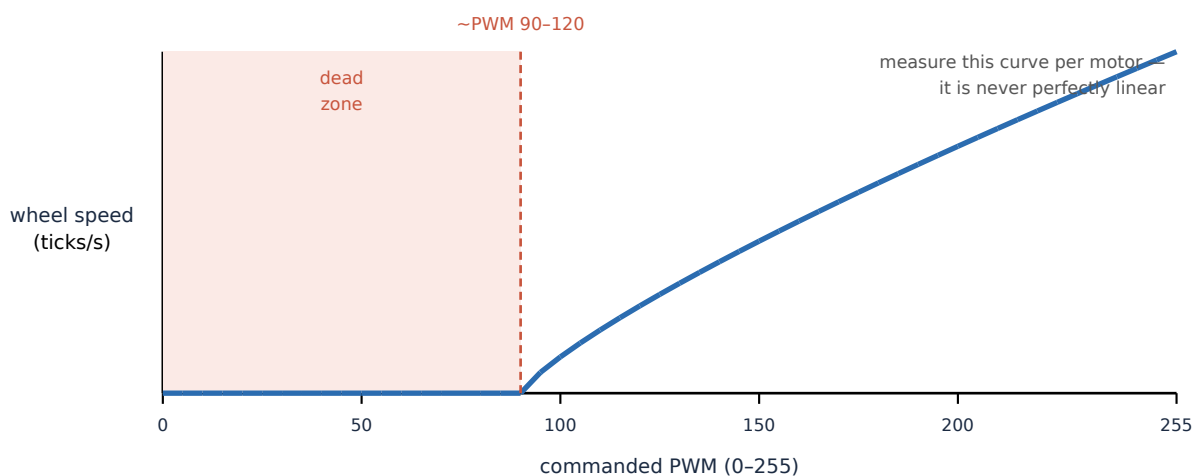


Figure 2 — Typical PWM-to-speed shape for a small brushed gear motor: flat until the dead zone is cleared, then rising, tapering slightly near maximum.

5. Design decisions and reasoning

5.1 Direction control and shoot-through protection

A standard 2-pin-per-motor direction scheme (IN1/IN2, IN3/IN4) is assumed, matching the driver style already assumed in the power document **[ASSUMPTION]**. Some H-bridge ICs can briefly cross-conduct (“shoot-through”) if the direction pins change state while PWM is still driving current — this can heat or, over many cycles, damage the driver. The rule is simple: **bring PWM to 0 before changing IN1/IN2, then ramp back up in the new direction**. This is the same rule the power document gives for the 5–8 A reversal current spike — here it protects the driver IC itself, not just the battery. **[RECOMMENDATION]**

5.2 Encoder decoding: single-edge now, quadrature-ready

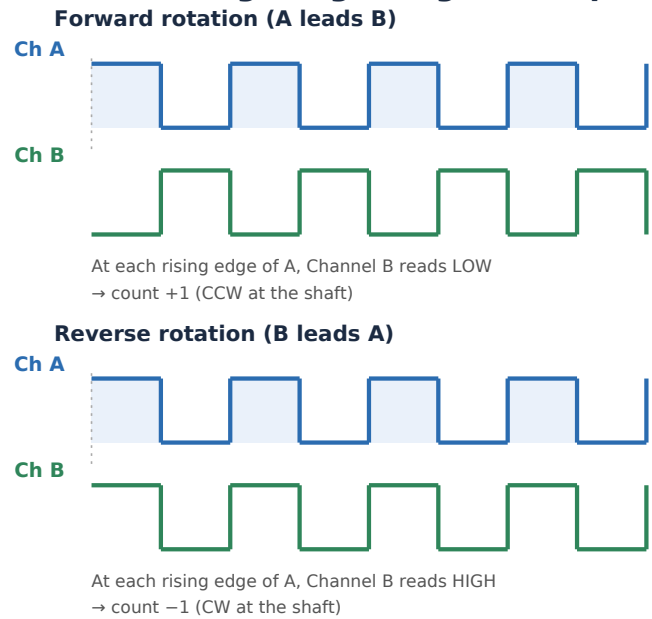


Figure 3 — How quadrature direction decoding works: which channel leads the other tells you the direction.

Every encoder pulse math in the team's existing guides counts **rising edges of Channel A only** — it measures how fast a wheel is turning, but not which way, so direction is assumed from the commanded PWM sign instead of measured. That is a reasonable, working starting point, and this document keeps it. Wiring Channel B to D11/D12 now, even though the firmware ignores it initially, means **full quadrature decoding is a firmware-only upgrade later** — useful if the robot needs to detect being pushed backward by a collision, or unexpected wheel slip during braking, neither of which a PWM-sign assumption can catch.

Scheme	What it detects	Cost	When to use
Single-edge, Ch A only (current default)	Speed magnitude; direction assumed from commanded PWM	1 interrupt pin per motor	Start here — matches existing team code, lowest risk
Full quadrature, both edges, both channels (4×)	Speed <i>and</i> true direction, 4× the resolution	Reads Ch B inside the Ch-A ISR (no extra interrupt pin, per Section 5.4)	Upgrade if backward-slip/collision detection is needed, or finer odometry resolution helps in Section B

5.3 Closed-loop speed control, not a fixed PWM ratio

The team's existing motor-characterization notes measure a fixed speed ratio between the two motors (e.g. “left is 93% of right”) and apply it as a constant correction. That works at one battery voltage and one load, but the real relationship shifts as the battery sags over the 30-minute access window (power document, Section 5) and as load changes (flat floor vs. the bridge incline).

[RECOMMENDATION] Use the encoder ticks themselves as live feedback: a small PID loop per wheel that targets a commanded ticks-per-control-cycle value, so PWM is continuously adjusted to hold the target speed regardless of voltage or load. The static ratio then becomes only a starting point for that PID's output, not the whole strategy.

5.4 Reading Channel B without a second interrupt pin

```
// Channel A ISR: counts pulses AND (optionally) reads direction from B
volatile long left_count = 0;
volatile int8_t left_dir = 1;    // +1 forward, -1 reverse (only meaningful once quadrature is enabled)

void leftEncoderISR() {
    // Reading digitalRead(B) here, right when A's edge fires, is the standard
    // way to decode direction without giving B its own interrupt pin.
    bool b = digitalRead(LEFT_ENC_B);
    left_dir = b ? -1 : 1;
    left_count += left_dir;      // starts as +1 always, until quadrature is turned on
}
```

5.5 Noise suppression

- **100 nF ceramic capacitor across each motor's two power terminals**, as close to the motor as practical — a standard, well-known fix for brush noise on small DC motors, and cheap insurance for the nearby encoder signal.
- **Keep encoder signal wires (yellow/green) bundled apart from motor power wires (red/white)** along their run, even though they share one JST-PH connector at the motor end — twist the encoder pair together where the run is long.
- **Route both motors' cables away from the buck regulator and battery leads** (power document, Section 6), for the same reason ultrasonic and IMU wiring avoid that area.

All of Section 5.5 is **[RECOMMENDATION]**.

6. How everything connects

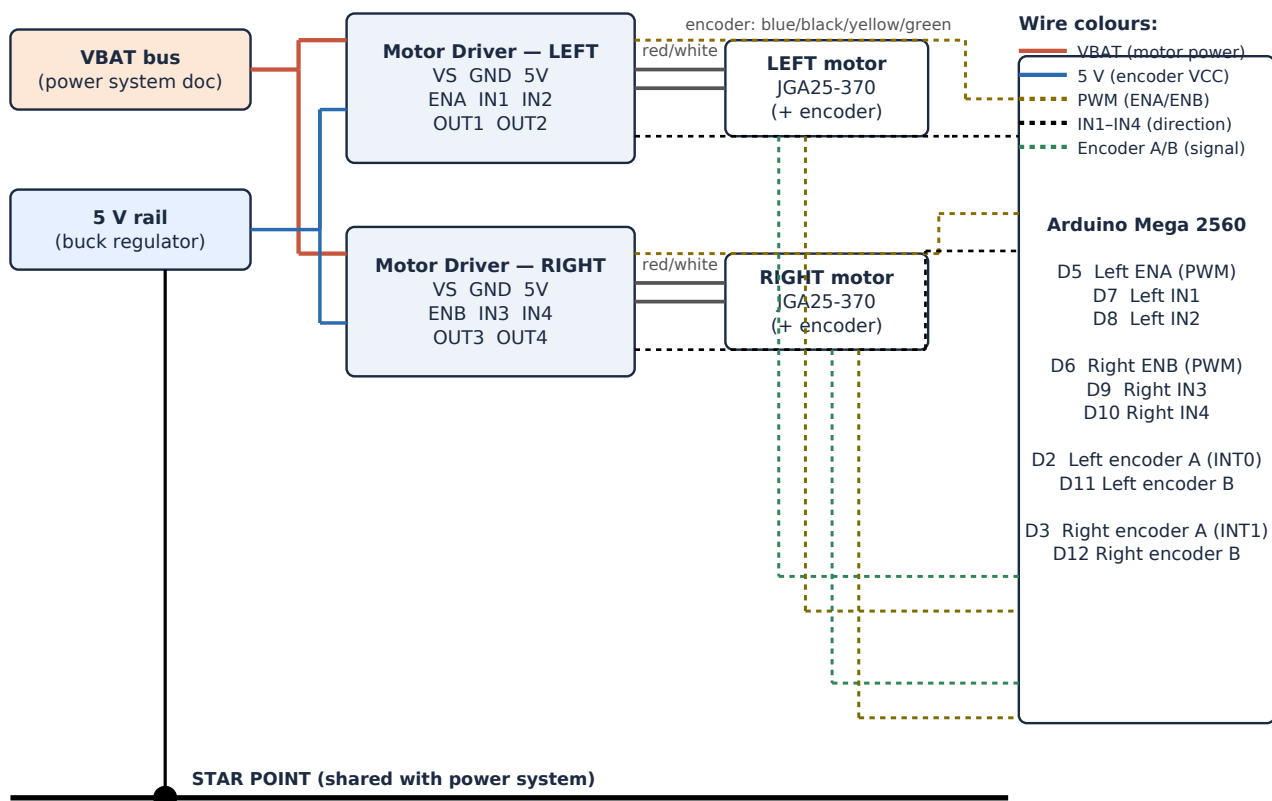


Figure 4 — Motor and encoder wiring. Red = VBAT, blue = 5 V, amber = PWM, black (dashed) = direction, green/olive (dashed) = encoder signal.

6.1 Connection table

From	To	Wire	Notes
VBAT bus (power document)	Driver VS (both drivers)	20 AWG	Same bus the motor drivers were already sized for
5 V rail (power document)	Driver logic 5 V, encoder VCC (blue) on both motors	22 AWG	Encoder current draw is a few mA per channel — negligible addition
Arduino D5 / D6	Driver ENA / ENB	signal	PWM speed
Arduino D7/D8, D9/D10	Driver IN1/IN2, IN3/IN4	signal	Direction
Driver OUT1/OUT2, OUT3/OUT4	Motor power leads (red/white)	20–22 AWG, twisted	Twisting reduces radiated noise
Motor encoder yellow / green	Arduino D2 or D3 (Ch A) / D11 or D12 (Ch B)	signal, twisted pair	Keep separated from the red/white power pair along the run
Motor encoder black	Star ground (power document)	signal	Not through the driver's motor-return path
Motor terminals	100 nF ceramic capacitor	—	Across the two motor power terminals, close to the motor

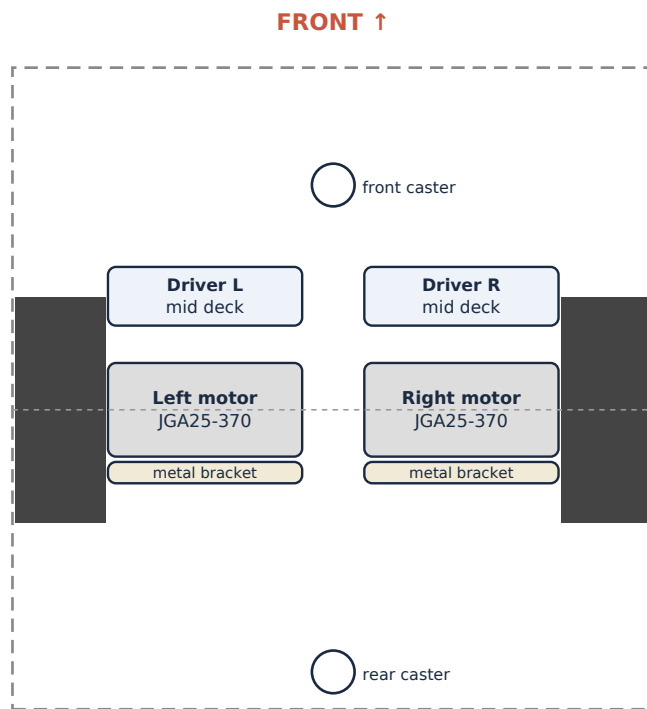
6.2 Wiring rules

- **Encoder signal wires are separated from motor power wires** along the whole run, not just at the connector — bundle yellow/green/blue/black together, and red/white together, even inside one JST-PH cable jacket where possible.
- **One shared ground.** Encoder GND (black) returns to the same star point as everything else (power document) — not through the motor driver's OUT/GND path, which carries motor switching noise.

- **Secure the JST-PH connector** at the motor end with a zip-tie or a dab of hot glue on the cable, not the connector body, so it can't shake loose; leave a small service loop rather than a taut wire.
- **No Dupont jumpers on the motor power path.** Screw terminals or soldered joints only, for the same reason given in the power document.
- **Label each cable** L/R at both ends — two visually identical motors make a swapped wire an easy, confusing mistake.

7. Placement on the chassis

The chassis and sensor/motor placement are the team's own 3D-printed design [\[SPEC\]](#); these drawings show the reasoning. [\[RECOMMENDATION\]](#)



Motors bolt to the metal wheel/motor brackets (provided), which set the wheel's position relative to the chassis — keep both sides symmetric so the axle stays perpendicular to the centreline.

Drivers sit just above each motor on the mid deck — shortest possible motor-power leads, away from IR/IMU signal wiring.

Route each motor's 6-wire encoder cable away from its own power leads and away from the other motor's leads — bundle encoder signal wires separately.

Leave a small service loop in each cable at the motor end so vibration doesn't pull on the JST-PH connector.

Figure 5 — Top view (schematic): motor, bracket and driver placement.

7.1 Placement rules

Part	Where	Why
Motors + metal brackets	Symmetric either side of the centreline, axle perpendicular to the chassis centreline	Any asymmetry becomes a permanent steering bias that closed-loop PID has to fight continuously
Motor drivers	Mid deck, immediately above each motor	Shortest possible high-current motor leads; keeps PWM switching noise away from the IR/IMU wiring runs on the other side
Encoder cable routing	Along the side of the chassis away from the buck regulator and battery leads	Same EMI reasoning as the power and ultrasonic documents
Cable service loops	A small loop of slack at each motor connector, not a taut wire	Vibration and chassis flex don't get transmitted straight into the connector
Weight balance	Motors ($\approx 300\text{--}400\text{ g}$ together) sit low, alongside the battery — both are among the heaviest parts on the robot	Keeps the centre of gravity low and helps traction on the bridge incline, consistent with the power document's placement rules

8. If you build a PCB

If the motor/encoder wiring is brought onto a custom PCB alongside the power system, treat the motor-power traces as high-current and everything else as signal-level. **[RECOMMENDATION]**

Item	Guidance
Motor power traces (driver OUT to motor)	Same treatment as motor traces in the power document — wide (≥ 3 mm or a pour) and short; these carry stall/reversal currents up to several amps
Encoder header	One 6-pin header per motor (matching the JST-PH pinout) or two smaller headers if power and encoder are split — signal-level traces (0.25–0.3 mm) are enough
Decoupling	100 nF pad at each motor's power header (Section 5.5); another 100 nF at each encoder VCC/GND pin pair
Keep-out	Route encoder signal traces away from driver OUT traces and PWM traces — cross at right angles if unavoidable, as with the ultrasonic document's motor keep-out rule
Ground	Tie encoder ground into the same pour as the rest of the board — do not give the encoders an isolated ground island
Test points	One per encoder Channel A/B (F/L/R-style labelling) — lets you verify pulse trains with a multimeter or logic analyser without desoldering
Silkscreen	Label L/R and A/B/VCC/GND at each header — the six near-identical wires are easy to mix up during assembly or repair

9. Software requirements linked to the motors

```
// Encoder ISRs (single-edge, Section 5.2 default)
volatile long left_count = 0, right_count = 0;
void leftEncoderISR() { left_count++; }
void rightEncoderISR() { right_count++; }

void setup() {
  pinMode(LEFT_ENC_A, INPUT_PULLUP);
  pinMode(RIGHT_ENC_A, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(LEFT_ENC_A), leftEncoderISR, RISING);
  attachInterrupt(digitalPinToInterrupt(RIGHT_ENC_A), rightEncoderISR, RISING);
}

// Safe direction change: coast to zero before flipping IN1/IN2 (Section 5.1)
void setMotor(uint8_t pwmPin, uint8_t in1, uint8_t in2, int target) {
  analogWrite(pwmPin, 0);          // stop switching current first
  delay(15);                       // let the H-bridge settle
  digitalWrite(in1, target >= 0 ? HIGH : LOW);
  digitalWrite(in2, target >= 0 ? LOW : HIGH);
  analogWrite(pwmPin, constrain(abs(target), 0, 255));
}

// Stall detection: PWM high but ticks flat (Section 4.4)
unsigned long stallSince = 0;
bool checkStall(int commandedPWM, long deltaTicks) {
  if (commandedPWM > 60 && abs(deltaTicks) < 2) {
    if (stallSince == 0) stallSince = millis();
    if (millis() - stallSince > 300) return true; // real stall, not a blip
  } else {
    stallSince = 0;
  }
  return false;
}
```

A per-wheel PID (Section 5.3) sits between these two pieces: it reads deltaTicks each control cycle, compares it against a target ticks-per-cycle value, and calls setMotor() with the corrected PWM — the existing PID guide in the project covers the controller itself; this document supplies the encoder feedback and the safe motor interface it runs on top of.

10. Bring-up and test checklist

#	Test	Pass condition
1	Continuity/polarity check, power off	No short across motor terminals; encoder wires not swapped with power wires
2	Each motor alone, ramped PWM 0→255	Spins smoothly; find and record the dead-zone PWM (Section 4.4)
3	Encoder pulse test, motor spinning	Ticks/sec increase monotonically with PWM; both encoders report the same order of magnitude
4	Quadrature check (if/when enabled)	Forward command gives consistent Channel-B phase; manually spinning the wheel backward flips the decoded direction
5	Speed-matching test	Both wheels hold the same commanded ticks/cycle under closed-loop PID, without needing a fixed PWM ratio
6	Deliberate stall (hold one wheel briefly)	Stall is detected within ~300 ms and motors are cut — release quickly, don't hold at stall current for more than a second or two
7	Direction-reversal test	No audible/visible driver distress; encoder direction (if enabled) tracks correctly through the reversal
8	Straight-line drift test	< 3% lateral drift over a measured run (team's existing threshold)
9	Bridge climb test at realistic PWM	Robot climbs the 35° ramp without stalling or excessive wheel spin — confirms the Section 4.3 torque margin for the real gear ratio
10	Endurance test, full 30-minute session	No mechanical loosening (bracket screws, JST-PH connectors) and no thermal issues on driver or motor casing

11. Recommended component lists

Motors, wheels, brackets and drivers are provided — get instructor approval before buying anything additional (spec p.7).

11.1 List A — Must have

#	Component	Specification	Qty	Source
1	DC encoder motors	Provided — JGA25-370, 12 V, with Hall encoder	2	Provided
2	Wheels + metal brackets	Provided, matched to the motors	2 each	Provided
3	H-bridge motor drivers	Provided — confirm exact model against assumption A1	2	Provided
4	100 nF ceramic capacitor	One per motor, across the power terminals	2	Capacitors THT category
5	Zip ties / cable clips	Strain relief for the JST-PH connectors	~6	Hardware shop

11.2 List B — Optional / nice to have

Component	Benefit
Small bench multimeter with a 10A+ current range	Measure real no-load/stall current per motor against Section 3's figures
Digital tachometer or a simple stroboscope app	Verify measured RPM at a given PWM against the datasheet range for your actual gear ratio
Spare driver module	Backup if one is damaged — requires instructor approval since it is not part of the provided set

11.3 List C — Extra parts for a custom PCB

Component	Purpose
2 × 6-pin header (matching JST-PH pinout)	Motor + encoder connection points
4 × 100 nF pads	Motor-terminal and encoder decoupling (Section 8)
4 × labelled test points	Probe encoder A/B per motor without desoldering

12. Viva preparation — questions you should be able to answer

- Why does encoder Channel A go on a true interrupt pin, but Channel B doesn't need one?
- Why did the encoder pins take priority over the ultrasonic ECHO pins in the master pin map?
- What is H-bridge shoot-through, and how does your code avoid it?
- Why is closed-loop (encoder-based) speed control better than a fixed PWM ratio between motors?
- How do you tell a genuine stall from the robot just slowing down briefly?
- What causes the PWM dead zone, and how do you compensate for it?
- Where does the 11 pulses/revolution figure apply — the motor shaft or the wheel? Why does that matter?
- How did you check whether the motors have enough torque to climb the bridge?